Data Structures

CSL202

Date: 23.10.2025 Quiz Set-B

Question 1: Insert the following keys into an initially empty AVL tree: {40, 25, 50, 10, 30, 60}

a) Draw the balanced AVL tree after all insertions (perform rotations as required).

b) Insert 55 and show the updated tree after rebalancing (mention rotation type).

c) Delete 25 and draw the final balanced AVL tree.

Question 2: Given the following unsorted elements: {42, 15, 27, 33, 18, 10, 5}

a) Build a max heap using bottom-up heapify and draw the heap tree after construction.

b) Build a max heap by inserting the elements one by one, showing the heap tree after each insertion.

Question 3: Given an array of n distinct integers, find the kth largest element using a heap-based

approach without sorting the array.

Input: arr = [7, 10, 4, 3, 20, 15], k = 4

Output: 7

Write pseudocode.(You may use a heap directly (e.g., max-heap or min-heap)

Data Structures

CSL202

Date: 23.10.2025 Quiz Set-B

Question 1: Insert the following keys into an initially empty AVL tree: {40, 25, 50, 10, 30, 60}

a) Draw the balanced AVL tree after all insertions (perform rotations as required).

b) Insert 55 and show the updated tree after rebalancing (mention rotation type).

c) Delete 25 and draw the final balanced AVL tree.

Question 2: Given the following unsorted elements: {42, 15, 27, 33, 18, 10, 5}

a) Build a max heap using bottom-up heapify and draw the heap tree after construction.

b) Build a max heap by inserting the elements one by one, showing the heap tree after each insertion.

Question 3: Given an array of n distinct integers, find the kth largest element using a heap-based

approach without sorting the array.

Input: arr = [7, 10, 4, 3, 20, 15], k = 4

Output: 7

Write pseudocode.(You may use a heap directly (e.g., max-heap or min-heap)

without implementing it from scratch.)

Data Structures

CSL202

Date: 23.10.2025 Quiz Set-A

Question 1: Insert the following keys into an initially empty AVL tree: {45, 20, 60, 10, 30, 70}

a) Draw the balanced AVL tree after all insertions (perform rotations as required).

b) Insert 65 and show the updated tree after rebalancing (mention rotation type).

c) Delete 20 and draw the final balanced AVL tree.

Question 2: Given the following unsorted elements: {55, 28, 37, 41, 22, 16, 9}

a) Build a max heap using bottom-up heapify and draw the heap tree after construction.

b) Build a max heap by inserting the elements one by one, showing the heap tree after each insertion.

Question 3: Given an array of n distinct integers, find the kth smallest element using a heap-

based approach without sorting the array.

without implementing it from scratch.)

Data Structures

CSL202

Date: 23.10.2025 Quiz Set-A

Question 1: Insert the following keys into an initially empty AVL tree: {45, 20, 60, 10, 30, 70}

a) Draw the balanced AVL tree after all insertions (perform rotations as required).

b) Insert 65 and show the updated tree after rebalancing (mention rotation type).

c) Delete 20 and draw the final balanced AVL tree.

Question 2: Given the following unsorted elements: {55, 28, 37, 41, 22, 16, 9}

a) Build a max heap using bottom-up heapify and draw the heap tree after construction.

b) Build a max heap by inserting the elements one by one, showing the heap tree after each insertion.

Question 3: Given an array of n distinct integers, find the kth smallest element using a heap-

based approach without sorting the array.

Input: arr = [7, 10, 4, 3, 20, 15], k = 3

Input: arr = [7, 10, 4, 3, 20, 15], k = 3

Output: 7

Write pseudocode.(You may use a heap directly (e.g., max-heap or min-heap) without implementing it from scratch.)

Data Structures

CSL202

Date: 23.10.2025 Quiz Set-C

Question 1: Insert the following keys into an initially empty AVL tree: {50, 35, 65, 25, 40, 75}

a) Draw the balanced AVL tree after all insertions (perform rotations as required).

b) Insert 45 and show the updated tree after rebalancing (mention rotation type).

c) Delete 35 and draw the final balanced AVL tree.

Question 2: Given the following unsorted elements: {68, 42, 53, 25, 36, 18, 12}}

a) Build a max heap using bottom-up heapify and draw the heap tree after construction.

b) Build a max heap by inserting the elements one by one, showing the heap tree after each insertion.

Question 3: You are given an array of integers representing weights of stones.

Output: 7

Write pseudocode.(You may use a heap directly (e.g., max-heap or min-heap) without implementing it from scratch.)

Data Structures

CSL202

Date: 23.10.2025 Quiz Set-C

Question 1: Insert the following keys into an initially empty AVL tree: {50, 35, 65, 25, 40, 75}

a) Draw the balanced AVL tree after all insertions (perform rotations as required).

b) Insert 45 and show the updated tree after rebalancing (mention rotation type).

c) Delete 35 and draw the final balanced AVL tree.

Question 2: Given the following unsorted elements: {68, 42, 53, 25, 36, 18, 12}}

a) Build a max heap using bottom-up heapify and draw the heap tree after construction.

You repeatedly smash the two heaviest stones together until at most one stone remains: ● If the two stones have equal weight, both are destroyed. ● If the two stones have different weights, the stone of smaller weight is destroyed, and the larger stone's weight becomes (difference of the two). Return the weight of the last remaining stone (or 0 if no stones remain) using a heap-based approach. Input: stones = [2, 7, 4, 1, 8, 1] Output: 1 Write pseudocode.(You may use a heap directly (e.g., max-heap or min-heap) without implementing it from scratch.)	b) Build a max heap by inserting the elements one by one, showing the heap tree after each insertion. Question 3: You are given an array of integers representing weights of stones. You repeatedly smash the two heaviest stones together until at most one stone remains: ● If the two stones have equal weight, both are destroyed. ● If the two stones have different weights, the stone of smaller weight is destroyed, and the larger stone's weight becomes (difference of the two). Return the weight of the last remaining stone (or 0 if no stones remain) using a heap-based approach. Input: stones = [2, 7, 4, 1, 8, 1] Output: 1 Write pseudocode.(You may use a heap directly (e.g., max-heap or min-heap) without implementing it from scratch.)